



GİT409U-DİJİTAL OYUN TASARIMI

Ünite 5: Varlık Davranışlarının Kontrolü

Giriş

Oyun geliştirmede varlıkların davranışlarını kontrol etmek önemlidir. Bu süreçte Unity oyun motoru ile C# programlama dili kritik bir rol oynamaktadır. C#, nesne yönelimli yapısı ve esneklik sağlayan özellikleriyle oyun geliştirme sürecinde geniş bir kullanım alanına sahiptir. C# kullanarak, oyunun ana yapısının oluşturulmasından, karakter hareketleri ve yapay zekâ davranışlarının geliştirilmesine, hatta oyun içi etkileşimlerin ayrıntılı şekilde düzenlenmesine kadar geniş bir alanda kontrol sağlanabilir.

Unity ve Script Kodlama

Script yani betik oluşturma süreci, Unity oyun geliştirme platformunun temel bileşenlerinden biridir. Unity platformu, oyunların davranışlarını ve mekaniklerini kontrol etmek için yaygın olarak C# programlama dilini kullanır. C# dili, geliştiricilere esneklik sağlayarak karmaşık işlevler oluşturma imkânı tanır. Ayrıca, kod yazmadan Script oluşturmak isteyenler için Görsel Programlama (Visual Scripting) yöntemi de mevcuttur. Bu yöntemle kullanıcılar, düğümleri arayüzde sürükleyip bırakarak program yapabilirler. Ancak bu ünite C# dili üzerinde durulmuş ve Unity ile etkili Script yazma tekniklerine odaklanılmıştır.

C# Programlama Dili

C# (C Sharp), Microsoft tarafından .NET platformu için geliştirilen, 2000'lerin başında Anders Hejlsberg önderliğindeki bir ekip tarafından tasarlanan, güçlü, esnek ve nesne yönelimli, Java ve C++ dillerinden ilham alarak, modern programlama gereksinimlerini karşılamak üzere geliştirilen bir programlama dilidir. Dil; LINQ, asenkron programlama ve daha birçok yenilikle zenginleştirilerek, sürekli olarak güncellenmiş ve evrimleşmiştir. C# içindeki temel yapı taşları:

- Değişkenler ve Veri Tipleri
- Operatörler
- Kontrol Yapıları
- Fonksiyonlar ve Metotlar
- Sınıflar ve Nesneler
- Arayüzler ve Soyut Sınıflar
- Kalıtım ve Polimorfizm
- Olaylar ve Delegeler

C# ve Unity'nin uyumu, oyun geliştirmede bu dilin popülerliğini artıran önemli bir faktördür. Unity'nin kullanıcı dostu arayüzü ve C#'ın esnekliği, geliştiricilere dinamik oyun mekanikleri, fizik motoru etkileşimleri, animasyon kontrolü ve yapay zekâ uygulamaları gibi güçlü yetenekler sunar. C#'ın nesne yönelimli yapısı, karmaşık oyun projelerinin yönetimini kolaylaştırır ve Unity ile etkili bir şekilde bütünleşir. Bu ikili, modern oyun geliştirme ihtiyaçlarını karşılayarak performans ve verimlilik sağlar.

Script Kodlama

Unity, kendi içinde C# kodlarını düzenlemek için bir IDE sunmaz. Ancak, Visual Studio, Visual Studio Code veya JetBrains Rider gibi harici kod editörleri kullanılabilir. Unity genellikle Visual Studio'nun ücretsiz sürümünü önerir ve bu sürüm, sf. 105 Görsel 5.1'de gösterildiği gibi Unity Hub üzerinden kolayca kurulabilir. Bu editörler, kod renklendirme, otomatik tamamlama ve hata ayıklama gibi özellikler sunarak geliştirme sürecini daha verimli hale getirir.

Başka bir IDE kullanmak istenirse, sf. 105 Görsel 5.2'de gösterildiği gibi Preferences bölümündeki External Tools seçeneğinden External Script Editör listesi aracılığıyla seçim yapılabilir. Unity, IDE'de yapılan kod değişikliklerini hızla tespit eder ve otomatik olarak derler. Bu hızlı senkronizasyon sayesinde, kodlar IDE'de derlenmeden bile Unity ile doğruluğu test edilebilir. Ayrıca, IDE'de yazılan kodlarda hata ayıklama yapılabilir; kesme noktaları eklenebilir, değişkenlerin değerleri izlenebilir ve yapıların içerikleri kontrol edilebilir.

Unity'de bir davranış modellemek için Script varlıkları oluşturulmalıdır. Bu Script varlıkları, bileşenlerin algoritmalarını ve davranışlarını yöneten dosyalardır. Özelleştirilmiş bileşenler oluşturmak için, MonoBehaviour sınıfından türetilmiş sınıflar yazılmalıdır. C# Script dosyaları, doğrudan Unity'nin proje yapılandırmasında oluşturulabilir. Script dosyasını eklemek için ilgili GameObject seçilir ve Inspector panelinde Add Component düğmesine basılarak New Script seçilir (sf. 106 Görsel 5.3). Script dosyasına tanımlayıcı bir isim verilmesi önemlidir. Oluşturulan Script, nesnelere eklenebilir ve böylece nesnelerin davranışları bu Script ile özelleştirilebilir. Bu adımlar, Unity'de dinamik ve interaktif oyun bileşenleri oluşturma temelinin oluşturur.

Unity'nin Project panelinde yeni oluşturulan Script dosyasının ismiyle bir varlık görüntülenir. Projelerin düzenini iyileştirmek ve dosyalar arasında daha rahat gezinebilmek için, tüm Script dosyalarını içeren "Scripts" adlı bir klasör oluşturulması tavsiye edilir. Eğer çok sayıda Script varsa, bu klasör içinde alt klasörler de oluşturulabilir. Bu düzenleme, projelerin büyümesiyle ortaya çıkan karmaşıklığı yönetmeyi ve düzeni korumayı kolaylaştırır. Ayrıca, Project panelinde yer alan Script dosyasına çift tıklayarak varsayılan olarak tanımlanmış IDE'de doğrudan açılabilir. Bu işlem, kodlama sürecine hızlı ve kesintisiz bir geçiş sağlar (sf. 107 Şekil 5.5). Bu yöntem, geniş projelerde kod organizasyonunu ve erişilebilirliğini artırarak verimliliği yükseltir.

Unity'de Script dosyaları, MonoBehaviour sınıfından türetilmiş sınıflar içerir ve bu sınıflar, GameObject'lerin yaşam döngüsünü ve davranışlarını tanımlar. Her GameObject için otomatik olarak bu sınıflardan bir nesne oluşturulur, böylece aynı Script dosyası ile birçok nesne yönetilebilir. Bu, geliştiricilerin her nesne için ayrı ayrı kod yazma yükünü ortadan kaldırır.



Olay Yönetimi

Unity'nin olay tabanlı programlama sistemi, geliştiricilere çeşitli durumlar için dinamik tepkiler oluşturma olanağı sunar. Bu sistem, belirli olaylar meydana geldiğinde, önceden tanımlanmış metotların çalışmasını sağlar. Örneğin, bir topun raketle çarpışması ya da bir nesnenin yok edilmesi gibi olaylar tetikleyici görevi görür ve bu olaylar gerçekleştiğinde, programcı tarafından belirlenen kod blokları devreye girer. Bu kod blokları, örneğin topun geri sekmesi gibi olay anında gerekli işlemleri yapar.

Unity'de nesnelerin davranışlarını kontrol etmek için sınıflarda olaylara yönelik metotlar tanımlanır. En yaygın olaylardan biri Update'tir. Update metodu, her kare yenilendiğinde otomatik olarak çalışır. Eğer bu metotta nesnenin konumunu değiştiren komutlar varsa, bu komutlar her karede çalışır ve nesnenin sürekli hareket ediyor gibi görünmesini sağlar.

Update metodunun doğru çalışıp çalışmadığını kontrol etmek için print fonksiyonu ile konsola mesajlar gönderilebilir. Yeni bir Script dosyası oluşturulduğunda, Update ve Start metodları sınıfa otomatik olarak eklenir. Update metoduna print("Update test"); komutu eklenerek bu metodun her karede çalışıp çalışmadığı test edilebilir. Start metodu ise yalnızca bir kez, GameObject sahneye eklendiğinde çalışır; bu nedenle print("Start test"); ifadesi sadece bir kez görünür. Oyun çalıştırıldığında, Unity Konsol penceresinde önce "Start test" ifadesi bir kere, sonra her kare yenilendiğinde "Update test" ifadesi görünür. Bu yöntem, Unity'nin olay yönetim sisteminin ve temel Script metotlarının işleyişinin basit bir testini sağlar. Ayrıca, çeşitli değişkenlerin değerleri de konsola yazdırılarak oyunun planlandığı gibi çalışıp çalışmadığı kontrol edilebilir.

Nesnelerin Hareketi

Unity'de bir nesnenin hareketini yönetmek için çeşitli metotlar bulunur. Bunlardan biri, nesnenin transform bileşeninin kullanımıdır. Transform bileşeni, nesnenin pozisyonunu, rotasını ve ölçeğini belirler ve bu bilgiler, nesnenin sahnedeki yerini ve yönünü belirlemek için önemlidir. Geliştiriciler, Inspector penceresi aracılığıyla bu transform değerlerini manuel olarak ayarlayabilir. Bununla ilgili örnek bir anlatımı kitabınızın 109. ve 110. sayfalarında inceleyebilirsiniz.

Bir nesneyi Unity'de hareket ettirmenin yollarından biri de kullanıcı girdisine dayalıdır. Oyun içindeki interaktiflik, oyuncunun girdi vermesiyle tetiklenir; örneğin, oyuncuların klavye üzerindeki W, A, S, D tuşlarına basmaları sonucunda nesne belirli bir yönde hareket ettirilebilir. Basit bir hareket mekanizması kurmak için WASD tuşlarına atanan hareketleri gösteren örnek kod bloğunu ve kodun işleyişini kitabınızın 111. sayfasında inceleyebilirsiniz.

Yeni Nesne Yaratma

Unity oyun motorunda, oyun sırasında dinamik olarak yeni nesneler yaratmak veya mevcut nesneleri yok etmek yaygın bir uygulamadır. Oyun başladığında veya ilerledikçe, oyuncunun eylemlerine tepki olarak yeni nesneler eklenebilir veya mevcut nesneler kaldırılabilir. Tüm oyun nesnelerini önceden oluşturmak verimsiz olabilir, bu yüzden nesneler ihtiyaç duyulduğunda oyun çalışırken (runtime) yaratılır. Unity, bu amaçla Instantiate fonksiyonunu sunar. Bu fonksiyon, örneğin bir tuşa basıldığında veya belirli bir zamanda prefab'ların örneklerini oluşturmaya imkân tanır. Prefab, bir nesnenin kalıbıdır ve nesnenin özelliklerini, bileşenlerini ve alt nesnelerini içerir. Prefab'ların yönetimi, yaratılan ve yok edilen nesneleri düzenlemeyi kolaylaştırır ve oyunun performansını ile oyuncu deneyimini iyileştirir. Bununla ilgili bir Prefab örneğini kitabınızın 113. sayfasında inceleyebilirsiniz.

Kamera Takip

Unity oyunlarında kamera kontrolü, oyuncu deneyimi için önemlidir ve genellikle oyun karakterini takip eden bir kameraya ihtiyaç duyulur. Karakteri takip eden bir kamera sistemi oluşturmak için, Main Camera'ya bir Script eklenir ve bu Script'te kameranın karakteri nasıl takip edeceği belirlenir. Bunun için, Hierarchy bölümünden Main Camera seçilir ve Inspector penceresinden yeni bir Script eklenir. Karakteri takip eden bir kamera için örnek bir kod bloğu ve işleyişi kitabınızın 116. sayfasında verilmiştir.

Çarpışma Yönetimi

Unity'de nesnelerin doğru şekilde çarpışmasını sağlamak için Collider bileşenleri kullanılır. Collider'lar, fizik motorunun hangi nesnelerin çarpışabileceğini ve hangilerinin geçilmez olduğunu anlamasını sağlar. Örneğin, bir oyun karakterinin duvarlardan geçmesini engellemek için karaktere uygun bir Collider eklenmelidir. Kapsül şeklindeki bir karakter için kullanılması gereken Capsule Collider'ı bir karaktere eklemek için takip edilmesi gereken adımlar şöyledir:

- Unity Editor'de Hierarchy panelinden karakter nesnesini seçin.
- Inspector panelinde, "Add Component" butonuna tıklayın.
- Physics kategorisinden "Capsule Collider"ı bulun ve seçin.
- Capsule Collider eklendiğinde, otomatik olarak nesnenin geometrisine uyacak şekilde boyutlandırılacaktır.
- Gerekirse, collider'ın boyutunu ve pozisyonunu manuel olarak ayarlayın.
- Collider'ın "Center" ve "Radius" özelliklerini ayarlayarak nesnenin gerçek dünya boyutlarına uygun hâle getirin.
- "Height" özelliğini, collider'ın yüksekliğini temsil edecek şekilde ayarlayın.



GİT409U-DİJİTAL OYUN TASARIMI

Ünite 5: Varlık Davranışlarının Kontrolü

- “Is Trigger” özelliğini, collider’ın bir tetikleyici olup olmadığını belirlemek için ayarlayın. “Is Trigger” etkinleştirilirse, collider diğer collider’larla çarpışmaları tetikleyici bir olay olarak kullanır fakat fiziksel bir engel oluşturmaz.

Unity’de Capsule Collider bileşeni bir karaktere eklendiğinde, o karakterin çevresini saracak beyaz çizgiler şeklinde bir sınır görüntüsü ortaya çıkar. Bu çizgiler Collider’ın fiziksel alanını ve etkileşim sınırlarını temsil eder. Bir karakter ya da nesne Capsule Collider ile donatıldığında, oyun içerisindeki diğer collider bileşenleriyle çarpışabilir ve nesnelerin içinden geçmesi engellenmiş olur. Collider’ların etkili bir şekilde çalışabilmesi için gerekli süreçler, bileşenler ve kod bloklarını kitabınızın 118. sayfasında inceleyebilirsiniz.

